

이광웅 - 경력기술서

Email: lgu4821@naver.com

Phone: 010-5852-9537

GitHub/Portfolio: KwangWoongLee/Portfolio (<https://github.com/KwangWoongLee/Portfolio>)

1. 경력 요약

C++ 기반 MMORPG 라이브 서버 개발자로 월드/존/유니버스 서버 콘텐츠, 대규모 PvP 콘텐츠, 거래소/상품/보상 시스템, 운영툴, DB 마이그레이션을 개발했습니다. 라이브 서비스에서 발생하는 이슈를 Grafana 지표, 서버 로그, 클라이언트 로그, DB 데이터로 추적하고 QA/PM/클라이언트 개발자/퍼블리셔 협업 부서와 재현 조건, 영향 범위, 검증 계획을 조율했습니다.

개인 포트폴리오에서는 실제 업무 경험과 연결되는 주제를 C++ IOCP 서버 구조, Actor 기반 상태 소유권, 공성전 상태 머신, DB 정합성, 병목 분석 흐름으로 재구성했습니다.

2. XLGAMES / Cube - 프로그램팀 서버파트

? 기간: 2023.06 ~ 현재

? 역할: C++ MMORPG 게임 서버 개발자

? 주요 환경: C++, MSSQL/T-SQL, Go, JavaScript, CMS, Protocol/CodeGen, GM Tool, Grafana, Git, Jira, Slack

2.1 주요 책임

? MMORPG 라이브 서비스의 월드/존/유니버스 서버 기능 개발

? 공성전, 렐름 공성전, 길드전, 권능의 신전 등 대규모 PvP 콘텐츠 서버 로직 및 운영툴 기능 개발

? 거래소, 웹상점, 상품 수령, 구매 검증 등 라이브 경제/상품 시스템 개선

? MSSQL 기반 운영 데이터 조회, 검증 쿼리, DB 마이그레이션 작성

? CMS/프로토콜/DB 변경을 수반하는 기능의 영향 범위 확인 및 데이터 정합성 검증

? Grafana, 로그, DB 데이터를 활용한 라이브 이슈 분석 및 협업 채널 공유

2.2 대규모 PvP 콘텐츠 서버 개발

배경

공성전, 렐름 공성전, 길드전, 권능의 신전처럼 다수 유저가 참여하는 콘텐츠는 전투 상태, 랭킹, 보상, 운영툴 제어, 서버 타입, 예외 상황에 따라 처리 경로가 복잡합니다. 라이브 서비스에서는 기능 구현뿐 아니라 강제 진행, 장애 상황, 보상 정합성, 운영툴 제어까지 함께 고려해야 합니다.

담당 및 구현

? 공성전, 렐름 공성전, 길드전, 권능의 신전 관련 서버 로직 개발/수정

? 전투 상태, 거점/기믹, 랭킹, 보상, 강제 진행 등 콘텐츠 상태 처리 구현

? 운영툴에서 공성전 제어, 상태 확인, 로그 조회가 가능하도록 기능 추가/개선

? 라이브 중 발생한 버그와 예외 케이스를 로그/DB/재현 조건 기반으로 분석하고 수정

협업 및 검증

? QA/PM/클라이언트 개발자와 재현 조건, 검증 범위, 영향 범위 공유

? 퍼블리셔 기술PM/QA와 라이브 이슈 제보, 로그 요청, 빌드 반영 일정 조율

? 서버 상태, DB 상태, 클라이언트 로그를 함께 확인해 콘텐츠 진행 중 발생 가능한 예외 상황 추적

포트폴리오 연결

개인 포트폴리오의 공성전 상태 머신, SiegeWar, SiegeManager, 참가 길드 잠금/해제 흐름은 이 경험을 개인 프로젝트에서 설명 가능한 형태로 재구성한 것입니다. 실제 라이브 콘텐츠 경험을 바탕으로 Scheduled -> Prepare -> InProgress -> Finished 상태 전이, 예외 취소, 타이머 호출, 종료 경계를 설계했습니다.

2.3 거래소/상품/결제 관련 라이브 시스템 개선

배경

거래소, 웹상점, 상품 수령 로직은 라이브 경제에 직접 영향을 주며, 서버 타입별 상품 노출 조건, 구매 제한, 만료 여부, 수수료/시세 계산, 수령 실패 재처리 같은 정합성 이슈가 중요합니다.

담당 및 구현

? 유니버스 거래소, 렐름 거래소, 웹상점, 상품 수령 로직 개발/수정

? 서버 타입별 상품 노출, 구매 가능 여부 검증, 만료 상품 구매 차단 처리

? 상품 수령 실패, 구매 검증 실패 등 라이브 이슈 원인 분석

? 거래소 수수료/시세 계산과 관련된 서버 로직 및 데이터 확인

? 상품/거래소/캐릭터/서버 상태 데이터를 MSSQL 쿼리로 조회하고 검증

포트폴리오 연결

포트폴리오에서는 공성 종료 후 보상 생성과 중복 방지 흐름을 DB 정합성 사례로 정리했습니다. 상품 수령/보상 지급 업무에서 중요했던 맥등성, 실패 재처리, 최종 방어선 관점을 개인 프로젝트의 작은 구조로 재구성했습니다.

2.4 운영툴 및 운영 자동화 기능 개발

배경

라이브 운영에서는 서버 기능 자체뿐 아니라 운영자와 QA가 상태를 확인하고 제어할 수 있는 도구가 중요합니다. 운영툴은 이벤트 우편, 서버 초기화, 공성전 제어, 로그 조회, 타임아웃 개선, 번역 적용 등 다양한 운영 업무와 연결됩니다.

담당 및 구현

? Go 기반 GM Tool/운영툴 백엔드 및 프론트엔드 기능 개발

? 이벤트 우편, 서버 초기화, 공성전 제어, 로그 조회 기능 개발/개선

? 운영 기록, 타임아웃, 번역 적용 등 운영/QA 효율 개선 작업 수행

? 서버 기능 변경에 맞춰 운영툴 API와 화면의 영향 범위 확인

포트폴리오 연결

현재 포트폴리오에서는 별도 운영툴 UI를 크게 만들기보다 서버 로그, 지표, 데모 로그, PDF 문서를 통해 운영 관찰성을 설명하는 방향으로 정리했습니다. 이후 공성 데모 로그와 Grafana/지표 캡처를 PDF에 포함하면 실제 운영 경험과 더 자연스럽게 연결됩니다.

2.5 DB/CMS/프로토콜 변경을 수반한 서버 기능 개발

담당 및 구현

- ? MSSQL 마이그레이션 작성
- ? 운영 데이터 조회/검증 쿼리 작성
- ? CMS 스키마 변경과 서버 로직 반영
- ? 프로토콜 구조 변경과 클라이언트/운영툴 영향 범위 확인
- ? 서버 기능 변경 시 DB, CMS, 프로토콜, 운영툴이 함께 변경되는 흐름 검증

포트폴리오 연결

포트폴리오에서는 캐릭터 저장 구조, 저장 프로시저, DB 완료 처리 경계를 통해 DB 정합성 책임 범위를 설명합니다. 실제 업무의 MSSQL/CMS/프로토콜 변경 경험을 개인 프로젝트에서는 더 작고 설명 가능한 구조로 재현했습니다.

2.6 라이브 이슈 분석 및 협업

담당 및 구현

- ? Grafana 지표, 서버 로그, 클라이언트 로그, DB 데이터를 함께 확인해 라이브 이슈 원인 분석
- ? QA/PM/클라이언트 개발자와 재현 조건, 영향 범위, 검증 계획 공유
- ? 퍼블리셔 기술PM/QA와 로그 요청, 이슈 공유, 빌드 반영 일정 조율
- ? 서버 상태와 DB 데이터를 기준으로 실제 영향 유저/상품/콘텐츠 범위 확인

포트폴리오 연결

포트폴리오의 SendOverflow/TCP 역압 분석, 단일 Zone 큐 병목 분석은 라이브 이슈 분석 역량을 개인 프로젝트로 보여주는 핵심 사례입니다. 단순히 병목을 주장하는 것이 아니라, CPU/메모리/네트워크/큐/패킷 지표를 보고 원인을 좁히는 과정을 PDF에서 설명합니다.

2.7 성과

- ? 재직 기간 중 연간 성과평가 우수 등급 2회 수령
- ? 대규모 PvP, 상품/거래소, 운영툴, DB/CMS/프로토콜 변경, 라이브 이슈 분석을 모두 경험하며 MMORPG 라이브 서버 개발자로서의 업무 범위를 확장

3. 프리메타 - 아프리카TV 자회사

- ? 기간: 2021.08 ~ 2022.08
- ? 역할: 서버 개발자
- ? 주요 환경: C++, IOCP, JavaScript, Node.js, React, MySQL 5.7, Redis, SVN, Windows Server, CentOS, Naver Cloud

3.1 주요 책임

- ? C++ 기반 IOCP 릴레이 서버 패킷 프로토콜 작업
- ? Node.js 기반 웹 서버 패킷 프로토콜 작업
- ? Node.js/React 기반 운영 관리툴 제작 및 관리
- ? Node.js 기반 Electron 패킷 툴 제작
- ? Naver Cloud 기반 서버 관리 지원
- ? Redis cluster/replica R&D 및 적용
- ? 아프리카TV 별 서비스 API 연동
- ? 게임 운영을 위한 서버 지원 전반

3.2 경험 요약

3개 서버군에서 메타버스 서비스의 런칭과 라이브 서비스 과정을 경험했습니다. 실시간 릴레이 서버, 웹 서버, 운영 툴, 패킷 툴, Redis, 클라우드 운영 지원을 함께 다루며 서비스 전체 구조를 넓게 이해하는 계기가 되었습니다.

3.3 현재 경력과의 연결

프리메타 경험은 이후 MMORPG 라이브 서버 업무에서 운영툴, 로그/패킷 확인, 서버와 운영 시스템 간 연동을 이해하는 기반이 되었습니다. 개인 포트폴리오에도 IOCP 서버 코어, 운영 관찰성, TestClient, 지표 흐름을 설계하는 데 이 경험을 반영했습니다.

4. 개인 포트폴리오 - C++ MMORPG 라이브 서버 포트폴리오

4.1 목적

완성된 MMORPG를 만드는 것이 아니라, MMORPG 라이브 서버 개발자가 실제로 설명해야 하는 구조와 문제를 개인 프로젝트로 표현하는 것이 목적입니다.

- ? 비동기 IO와 패킷/게임 로직 분리
- ? Actor 단위 상태 소유권과 경쟁 상태 대응
- ? Player 내부 모델의 변경 순서를 PlayerActor가 소유하는 구조
- ? Zone 병목을 지표로 확인하는 과정
- ? 장시간 콘텐츠 상태 머신과 타이머 호출
- ? DB 요청, 트랜잭션, 멍등성 경계
- ? 운영 로그와 장애 재처리 설계

4.2 구현 요약

- ? Windows IOCP 기반 비동기 TCP 서버
- ? GameLogic, NetworkIO, DB, Timer 실행기 분리
- ? Player, Zone, World, SiegeWar 단위 키 직렬 Actor 실행
- ? NetworkIO, Timer, DB 완료 경로가 상태를 직접 바꾸지 않고 Actor 메시지를 만드는 구조
- ? Zone 입장 예약, 정리 중 상태, 스냅샷 캐시
- ? 공성전 Scheduled/Prepare/InProgress/Finished/Canceled 상태 머신
- ? 공성 선포 비용 결제/타임아웃/환불 메시지 흐름
- ? 선포 예약 성공 시 참가 길드 잠금, 결제 실패/타임아웃/종료 시 해제
- ? 공성 종료 후 보상 대상 산정과 중복 방지 흐름 정리
- ? 캐릭터 단위 보상 적합성을 DB 사례로 정리
- ? DbCompletionTarget 기반 DB 완료 처리 경계
- ? GoogleTest 기반 공성전 상태 전이와 핵심 규칙 검증

4.3 병목 분석 흐름

초기 부하 테스트에서 SendOverflow가 발생했지만, 원인을 서버 송신 로직으로 단정하지 않고 클라이언트 수신 처리량, TCP 역압, 대기 송신 바이트, 워커 큐를 함께 확인했습니다. 클라이언트 프로세스 분산 후 3000봇까지 안정화 되었고, 4000봇에서는 CPU가 낮아도 단일 Zone 큐가 증가하는 서버 병목을 확인했습니다.

이 흐름은 실제 라이브 이슈 분석처럼 현상 관찰 -> 지표 확인 -> 가설 분리 -> 개선 -> 남은 병목 확인의 순서로 설명할 수 있게 정리했습니다.

5. 핵심 강점

- ? 대규모 PvP 콘텐츠 경험이 포트폴리오의 공성전 상태 머신/보상 설계로 이어진 점
- ? 상품/보상/거래소 경험이 DB 정합성과 중복 방지 설계로 이어진 점
- ? 운영툴/Grafana/로그 기반 이슈 분석 경험이 지표 기반 병목 분석으로 이어진 점
- ? C++ IOCP와 Actor 구조를 단순 구현이 아니라 상태 소유권과 실행 순서를 설명하기 위한 구조로 설계한 점
- ? 측정 결과를 바탕으로 워커 설정, 틱 배치, Zone 분할 같은 개선 후보를 구체화한 점